

Laboratorio Semana 8 – Cifrado AES-256, HMAC-SHA256, SSH y Git

Nombre: Pablo David Molina Son

Objetivo Aplicar cifrado simétrico para proteger la confidencialidad de un archivo, utilizar HMAC-SHA256 como huella autenticada y trabajar con un repositorio compartido de GitLab mediante SSH.

Herramientas: PowerShell + Git + OpenSSL

Regla de seguridad: Nunca subas la contraseña o clave secreta al repositorio de GitLab.

1. Conexión al repositorio mediante SSH

Verifica Git: `git --version`

Prueba la conexión: `ssh -T git@github.com`

Si es la primera conexión, verifica el fingerprint del servidor antes de aceptarlo.

Clona el repositorio compartido: `git clone git@github.com:[tu_repo]_git`

Entra al repositorio y crea tu rama: `cd git checkout -b lab-[nombre_branch]`

```
[pdms@000 ~]$ git --version
git version 2.55.0
[pdms@000 ~]$ ssh -T git@github.com
Hi pdmolinas! You've successfully authenticated, but GitHub does not provide shell access.
[pdms@000 ~]$ cd Documents/url-mia-2026/github/
[pdms@000 github]$ git checkout -b lab-8
Switched to a new branch 'lab-8'
[pdms@000 github]$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
[pdms@000 github]$ git checkout -b lab-8-Pablo-Molina
Switched to a new branch 'lab-8-Pablo-Molina'
[pdms@000 github]$ git branch -r lab-8
fatal: the -a, and -r, options to 'git branch' do not take a branch name.
Did you mean to use: -a|-r --list <pattern>?
[pdms@000 github]$ git branch -d lab-8
Deleted branch lab-8 (was 274b6a9).
[pdms@000 github]$ git branch
* lab-8-Pablo-Molina
main
[pdms@000 github]$ |
```

2. Crear un archivo personalizado

Creas un archivo cuyo nombre incluya tu nombre y apellido. Cada estudiante debe utilizar contenido personalizado.

Ejemplo: `mensaje_LuisRojas.json`

Contenido:

```
{
  "nombre": "TU NOMBRE",
  "curso": "Manejo e Implementación de Archivos",
  "carne": "[tu_carne]",
  "archivo": "mensaje_[Nombre_apellido].txt"
}
```

3. Cifrar el archivo con AES-256

Utiliza **OpenSSL** para cifrar tu archivo con **AES-256-CBC**. El archivo resultante tendrá extensión .enc

3.1 Verifica OpenSSL

Ejecuta el siguiente comando en **PowerShell**: `openssl version`

Si OpenSSL no está instalado, ejecuta: `winget install --id ShiningLight.OpenSSL.Light --exact`

Si instalaste OpenSSL, **reinicia VS Code** antes de continuar.

3.2 Comprueba instalación

`Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

3.3 Cifra tu archivo

Comprueba dónde quedó instalado: `Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

Agrega OpenSSL al PATH de tu usuario: `$opensslPath = "C:\Program Files\OpenSSL-Win64\bin"`

`[Environment]::SetEnvironmentVariable("Path", $env:Path + ";" + $opensslPath, "User")`

Actualiza el PATH de esta terminal inmediatamente: `$env:Path += ";C:\Program Files\OpenSSL-Win64\bin"`

3.4 Cifra tu archivo

Ejecuta: `openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.json -out .\mensaje_TuNombre.enc`

Cuando OpenSSL solicite la contraseña, introduce una contraseña que solo tú conozcas. Deberás introducirla dos veces. **NO escribas la contraseña en este laboratorio ni la subas a GitHub.**

```
[pdms@000 lab8]$ openssl --version
OpenSSL 3.6.3 9 Jun 2026 (Library: OpenSSL 3.6.3 9 Jun 2026)
[pdms@000 lab8]$ ls
mensaje_PabloMolina.json
[pdms@000 lab8]$ openssl enc -aes-256-cbc -in mensaje_PabloMolina.json -out mensaje_PabloMolina.enc -pbkdf2
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
[pdms@000 lab8]$ |
```

3.5 Comprueba que se creó el archivo: Ejecuta: `Get-ChildItem`

Deben aparecer, como mínimo: `mensaje_TuNombre.txt` `mensaje_TuNombre.enc`

```
[pdms@000 lab8]$ ls
mensaje_PabloMolina.enc  mensaje_PabloMolina.json
[pdms@000 lab8]$ |
```

3.6 Comprueba que el archivo cifrado no muestra directamente el mensaje:

Get-Content .\mensaje_TuNombre.enc

¿Qué observaste?

```
[pdms@000 lab8]$ ls
mensaje_PabloMolina.enc mensaje_PabloMolina.json
[pdms@000 lab8]$ cat mensaje_PabloMolina.enc
Salted__!~0%#_NR0000}&$0J00"i0X000020?,ø%020뵆 xz0000z]r5E0T0V000f000\0S,      000I00000T0
00-00
00hP0}00J000W$n0ëw=000: k}00w000!00T0H[pdms@000 lab8]$ |
```

Observo el archivo de forma encriptada.

4. Descifrar y recuperar el archivo

Utiliza la misma contraseña para recuperar el contenido original.

openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.enc -out

.\mensaje_TuNombre_recuperado.json

Comprueba el contenido recuperado: Get-Content .\mensaje_TuNombre_recuperado.json

```
[pdms@000 lab8]$ ls
mensaje_PabloMolina.enc mensaje_PabloMolina.json
[pdms@000 lab8]$ openssl enc -d -aes-256-cbc -in mensaje_PabloMolina.enc -out mensaje_PabloMolina_recuperado.json -pbkdf2
enter AES-256-CBC decryption password:
[pdms@000 lab8]$ ls
mensaje_PabloMolina.enc mensaje_PabloMolina.json mensaje_PabloMolina_recuperado.json
[pdms@000 lab8]$ cat mensaje_PabloMolina_recuperado.json
{
  "nombre": "Pablo Molina",
  "curso": "Manejo e Implementación de Archivos",
  "carne": "1338825",
  "archivo": "mensaje_PabloMolina.txt"
}
[pdms@000 lab8]$ |
```

5. Generar una huella autenticada con HMAC-SHA256

Ahora generarás una huella que depende del archivo y de una clave secreta. Esto permite comprobar integridad y autenticidad cuando quien verifica también conoce la clave.

archivo + clave secreta → HMAC-SHA256 → huella

corre: openssl dgst -sha256 -hmac "TU_CLAVE_SECRETA" .\mensaje_LuisRojas.json

¿Cuál es el HMAC-SHA256 obtenido?

```
[pdms@000 lab8]$ openssl dgst -sha256 -hmac "Bl@ck0ut" mensaje_PabloMolina.json
HMAC-SHA2-256(mensaje_PabloMolina.json)= b029ec8c31f2d0e55dbfbb754013e3ed3ce143b74f67eb853aa68803ada0dd1e
[pdms@000 lab8]$ |
```

6. Hash vs. cifrado vs. HMAC

Mecanismo	Objetivo	Clave	Ejemplo
SHA-256	Integridad	No	Huella
AES-256 (encrypt)	Confidencialidad	Sí	Archivo .enc
HMAC-SHA256	Integridad + autenticidad	Sí	Huella autenticada
SSH	Comunicación/autenticación segura	Sí	GitLab

Pregunta 1: Si alguien obtiene tu archivo .enc pero no conoce la contraseña, ¿qué propiedad estás protegiendo?

El contenido del archivo

Pregunta 2: ¿Por qué no debes subir la contraseña a GitLab?

Porque eso revelaría el archivo que estamos encriptando.

Pregunta 3: ¿Qué diferencia principal existe entre SHA-256 y HMAC-SHA256?

Que una es usada para generar una huella y la otra es para una huella autenticada.

7. Entrega

Dentro del repositorio de GitHub, crea la carpeta semana-8/lab8/ y coloca en ella un PDF con la solución del laboratorio, incluyendo las respuestas, el HMAC-SHA256 obtenido y pantallazos como evidencia de los comandos y procedimientos realizados. También incluir el archivo .json creado y el archivo .enc generado durante el cifrado. Enviar en la plataforma el enlace de la carpeta lab8 como evidencia de la entrega.